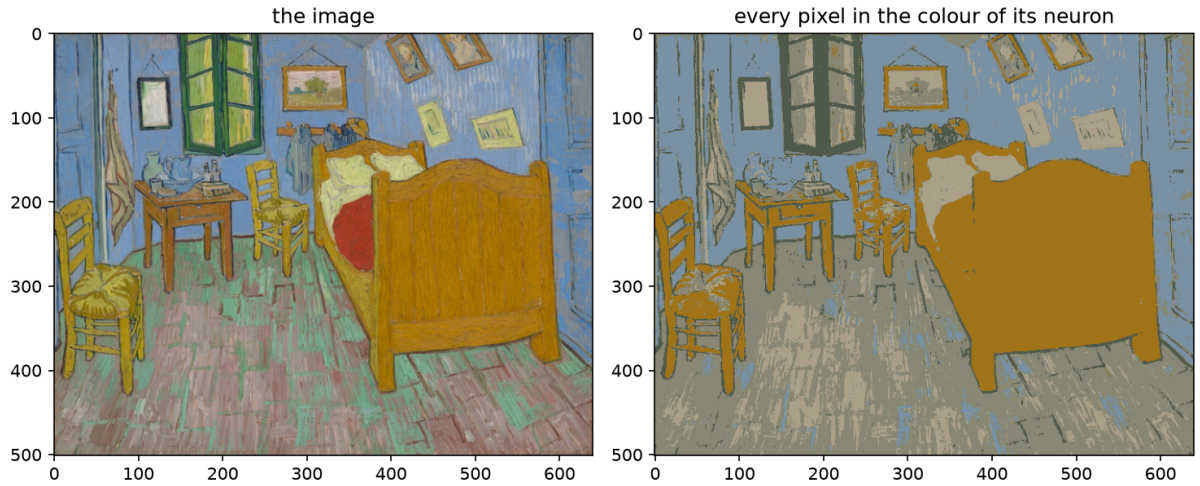
CS 301 · MEETING 12 · RECAP 1

What Big O Counts

Operation counts, growth rates, and timing estimates, using the pre-class notebook and your measurements.

One line, three ways

```
S   = X @ W.T + b                                          # NumPy, in the CPU's memory
S_t = X_t @ W_t.T + b_t                                    # PyTorch, on the CPU
X_d, W_d, b_d = X_t.to("cuda"), W_t.to("cuda"), b_t.to("cuda")   # every operand into the GPU's memory
S_d = X_d @ W_d.T + b_d                                    # on the GPU; S_d.cpu() brings it back
```



- X (n, m), W (K, m), b (K,): the scores S have shape (n, K)
- All tensors in an operation must use the same device

Count the operations

$$n \times K \times m$$

n	m	K	passes, n x K x m	seconds	ratio to the first	nanoseconds per pass
400	30	20	240,000	0.086	1.0	358
800	30	20	480,000	0.172	2.0	358
800	60	40	1,920,000	0.661	7.7	344

- ×2 when n doubles; ×8 when n, m and K all double
- Iterations: nKm, summarized as **O(nKm)**
- n = m = K = size: doubling size gives ×8 iterations

The count of every operation of this unit

operation	in NumPy	count	O( )	every size ×2
one neuron, one point	`w @ x + b`	2m	O(m)	×2
one neuron, n points	`X @ w + b`	2nm	O(nm)	×4
a layer of K neurons, n points	`X @ W.T + b`	2nKm	O(nKm)	×8
two matrices of scores, added	`A + B`	nK	O(nK)	×4
the softmax of every row	`softmax_rows(S)`	a few × nK	O(nK)	×4
a transpose, a reshape that returns a view	`W.T`, `X.reshape(...)`	metadata only	O(1)	×1

- O( ): how the count grows; the constant in front is dropped
- A transpose creates a view without copying: later access costs depend on the operation (Recap 2)

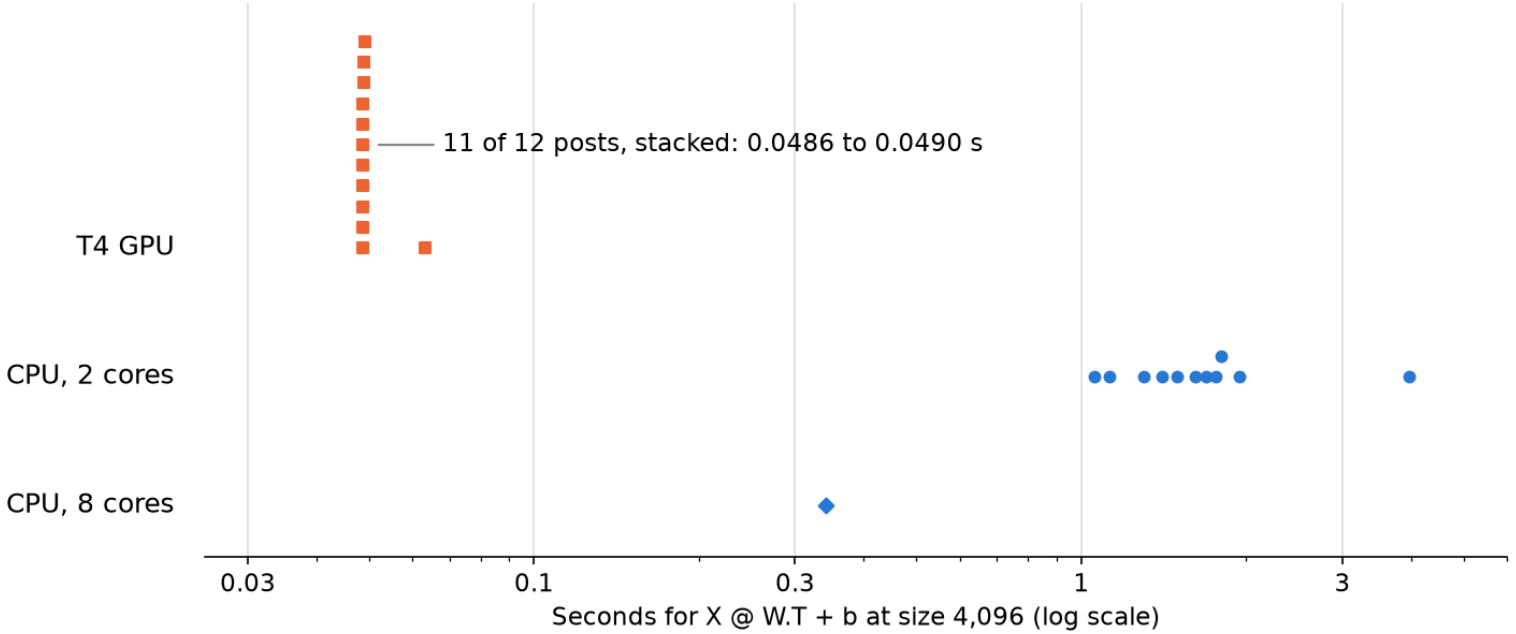
The cube is not an exponential

size^3: $\times 8$ when the size doubles $\qquad$ 2^{size}: $\times 2$ when the size grows by one

size	512	1,024	2,048	4,096
size^3	134 million	1.07 billion	8.6 billion	68.7 billion
2^{size}	155 digits	309 digits	617 digits	1,234 digits

- Power-law growth gives straight lines on log-log axes
- An exponential bends upward on the same axes

Your posts: the product at size 4,096

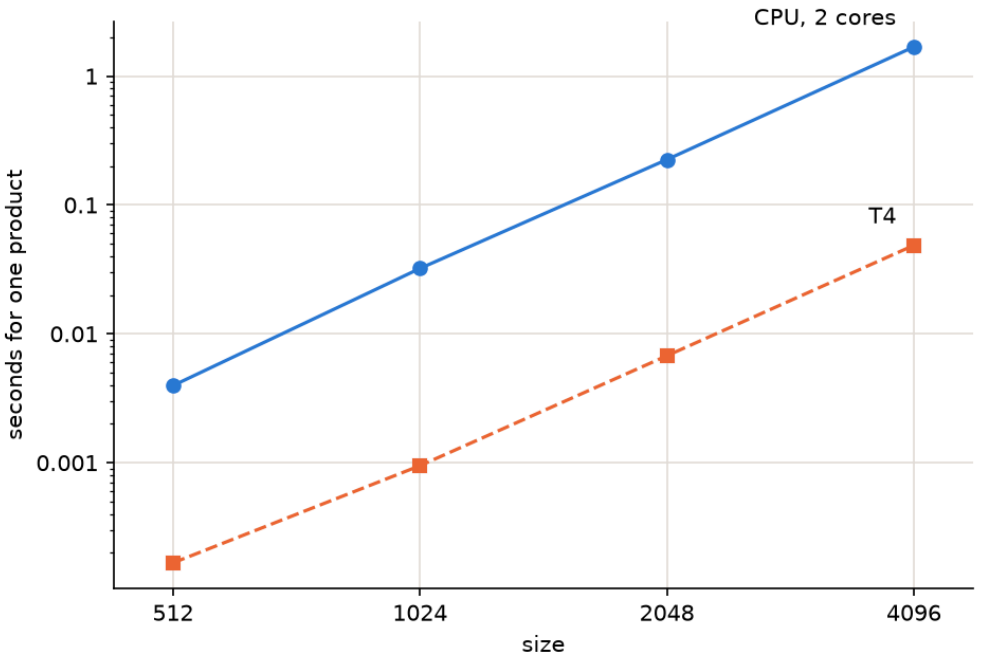


- The same count on every machine: 137 billion operations
- The T4: **0.049 s** in 11 of 12 posts; Colab's CPU: 1.1 to 1.9 s, one at 4.0
- GPU speedup: **22–40×**, one result at 63×; median 33×

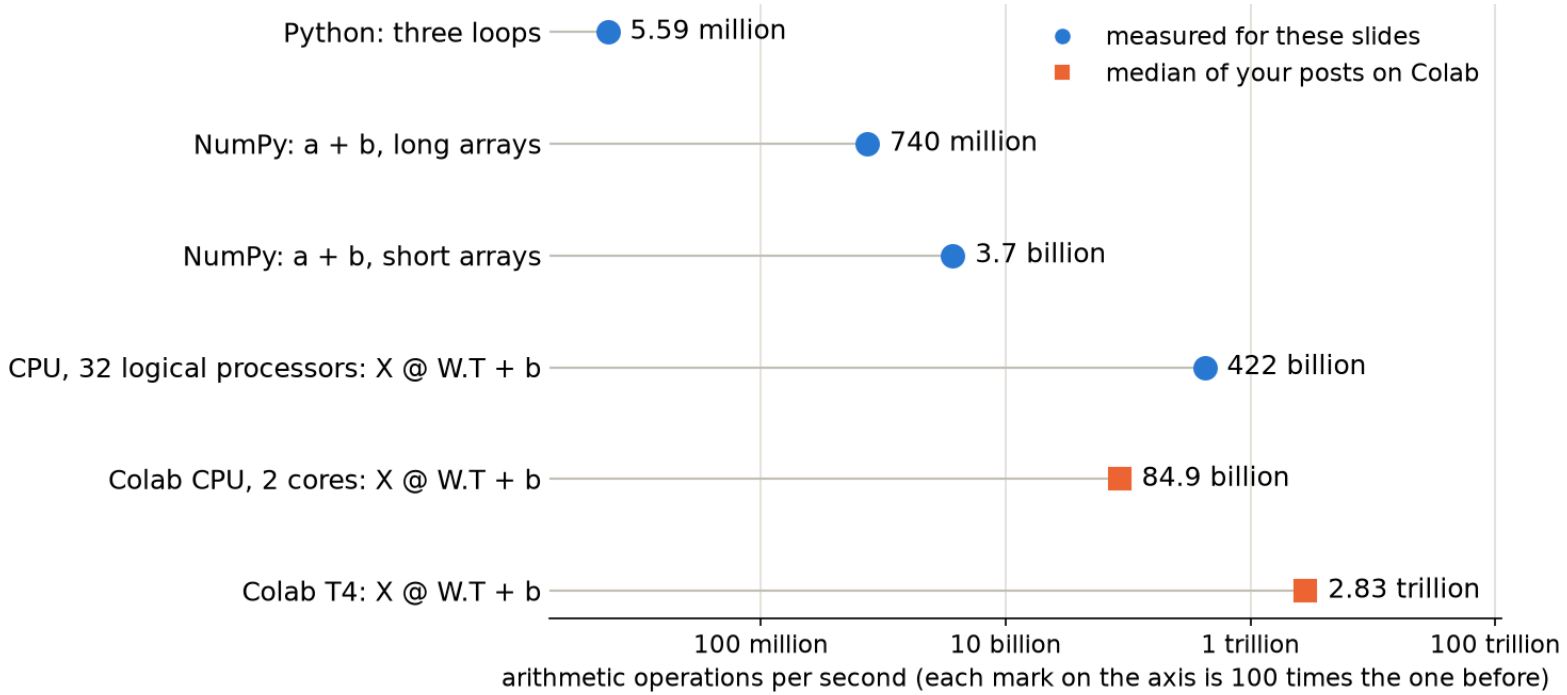
One group's whole table

size	CPU, s	T4, s	CPU / T4
512	0.0040	0.00017	24
1,024	0.0322	0.00095	34
2,048	0.2251	0.0068	33
4,096	1.6865	0.0486	35

- 2,048 → 4,096: ×7.5 on the CPU, ×7.2 on the T4: the count's ×8 on both
- Similar growth rates; the T4 is about 34 times faster



Measured operations per second



- Measured rates range from millions to trillions of operations/second
- Blue: measured for these slides (32 logical processors). Orange: your posts on Colab

Predict, then measure

$$\text{time} \approx \frac{\text{number of operations}}{\text{operations per second}}$$

```
K = 1,000 neurons, n = 60,000 images of 28 x 28 pixels (m = 784): 94.1 billion operations
implementation                          predicted time
Python: three loops                          4.67 hours
CPU, 32 logical processors: X @ W.T + b  223 milliseconds
Colab CPU, 2 cores: X @ W.T + b            1.11 seconds
Colab T4: X @ W.T + b                   33.3 milliseconds
```

- Predicted times: hours, about a second, or 30 milliseconds
- Second post: compare your predicted and measured layer times

What O( ) leaves out

$$\text{time} \approx \text{count} \times \text{seconds for one operation}$$

- **Operation count**: O( ) describes its growth
- **Time per operation**: varies with the implementation and hardware
- Relevant factors: interpreter overhead, access order, memory, and processor